FINTECHATHON 2026 · INTERNATIONAL TRACK · TOPIC E

Security Self-Assessment

FarmLink AI · Team FarmLink, American International University-Bangladesh · Permission tiers, controls, test evidence and known risks

Scope: FarmLink AI competition prototype (guided sandbox on GitHub Pages, plus the optional live sandbox API on Render). It handles **no real money, identities, lenders or credentials**. This document covers the controls that are actually implemented, the tests that exercise them, and the risks that remain.

1. Permission tiers (implemented)

Tier	Who / what	Can do	Cannot do	Enforced by
T0 Public visitor	Anyone with the URL	Use guided mode in their own browser; call `GET /v1/health`; create an anonymous session (rate-limited)	Read any session's data	No data is served without a session; per-IP and global session quotas
T1 Session holder	Holder of a bearer capability (32 random bytes; only its SHA-256 hash is stored)	Read and write **their own** workspace; run analyses; create drafts; prepare actions; confirm their own pending actions; export their evidence; delete their session	Access other sessions; bypass expiry; exceed quotas	`requireSession` middleware; every DB operation runs in a transaction scoped to `req.session.id`; the client-supplied workspace ID is ignored
T2 Model (agent)	Gemini, acting inside a T1 session	Call 8 allowlisted tools: `get_business_records`, `calculate_cashflow`, `match_financing_options`, `run_stress_test`, `create_application_draft`, `prepare_sandbox_submission`, `get_application_status`, `export_execution_log`	Confirm, approve, transfer, submit, fetch URLs, run SQL/shell/code, act on another session, or exceed 3 round trips / 6 tool calls per turn	Allowlist check, then zod argument validation, then execution through the same service code as the UI. Unknown tools are rejected and logged. The loop stops as soon as a pending action exists.
T3 Human confirmation	The T1 user pressing **Confirm** after ticking the review checkbox	Move one `AWAITING_CONFIRMATION` action to `SANDBOX_SUBMITTED`	Confirm a stale, expired, cancelled or mismatched action	`POST /v1/actions/:id/confirm` (not in the tool registry). It reloads the action and draft, checks expiry, versions and the recomputed payload hash, requires an `Idempotency-Key`, and commits atomically under a row lock
T4 Operator	Repository/Render owner	Set env vars, the kill switch (`LIVE_AI_ENABLED`) and quotas	n/a	Render dashboard; secrets are never in the repo or the bundle

2. Controls by threat

Threat	Control	Evidence
Leaked provider key	The key exists only in the backend env. There are no `VITE_*` secrets, and CI secret-scans both the source and `frontend/dist`. `/v1/health` exposes no config values.	`health exposes no secrets…` test; `scripts/secret-scan.mjs` in CI
Cross-session access (IDOR)	Session-scoped transactions; unknown IDs return 404 without revealing whether they exist	`another session cannot read or confirm…`
Forged authority (`approved=true`, client hash, role)	Ignored; confirmation re-derives everything server-side	`requires an idempotency key and ignores forged approval flags`; `ignores client-supplied workspace ownership fields`
Model confirms or escalates on its own	No such tool exists. Unknown tools are rejected before execution. The loop halts on a pending action.	`rejects unknown tools (including an attempted confirm)…`; `stops immediately after preparing…`
Prompt injection in records	Records are passed as `untrusted_description` data, the system prompt says to treat them as data, and the tool boundary holds even if the model obeys the injection	`prompt injection inside record text does not cause unauthorized actions`
Malformed model arguments	zod `.strict()` schemas and an ID regex	`rejects invalid tool arguments before execution`
Double-click / lost-response replay	`Idempotency-Key` plus a row lock (Postgres `SELECT … FOR UPDATE`; per-session mutex in memory). The same key returns the same receipt; a different key gets 409.	In-memory and Postgres concurrency tests
Change after review	Payload hash, draft version, workspace version and catalog version are bound. Any edit invalidates the action.	`a change after review makes the action stale`; local-client tests
Human confirms a fragile option without seeing the risk	The review dialog stress-tests the exact frozen terms; when they are not resilient it lists the failing scenarios and requires a second acknowledgement. `prepare_sandbox_submission` returns a `stressCheck` that tells the model to warn the user.	`prepare tells the model the stress result…`; guided e2e flow
Expired action / session	Server-side `expiresAt` check on every request, independent of cleanup	`expired actions are denied`; `expired sessions are denied…`
Cost abuse of a public model proxy	Kill switch; per-session hourly run quota; global daily provider-call cap reserved atomically **before** each call (not refunded on ambiguous failures); per-IP rate limits; message/history/body size limits; 25 s provider timeout	`exhausted quota…`, `global provider budget…`, `kill switch…`
Model outage	Returns 503 with the run ID. The run is stored as `failed`. No reply is invented.	`model outage returns an error without inventing a response`
XSS via records or model text	React renders text only; no `dangerouslySetInnerHTML` and no remote images; the API returns JSON	`stores HTML in descriptions as inert text`; code review

Threat	Control	Evidence
Silent currency mixing	Products whose currency differs from the workspace currency are `does_not_fit / CURRENCY_MISMATCH`. Display conversions are labeled approximate and never enter calculations.	engine `never compares products across currencies`
CSV formula injection	Export prefixes `= + - @ \t \r` cells with `'`	`neutralizes spreadsheet formula injection on export`
Oversized / malformed input	1 MB JSON limit, 415 for non-JSON, zod validation, 1,000 records and 1 MiB CSV per workspace	`rejects oversized payloads and non-JSON content types`
CORS misuse	Exact-origin allowlist, no credentials, no wildcard (CORS is not treated as authentication)	`CORS allows only the exact configured origin`
Data durability	Postgres; Cloud/Render disk is never used as the only copy	`commits one submission under parallel confirmation and survives a restart` (real Postgres 16)
Headers	`helmet` defaults; `x-powered-by` disabled	Configuration

3. Known risks and limits (honest list)

1. **A bearer token is not identity.** Whoever holds the session token *is* that session. UI confirmation proves only that a request came with that token, not that a particular human pressed the button; a script holding the token could call the confirm endpoint. That is acceptable for a sandbox with no real-world effect, but it is not strong customer authentication, KYC or non-repudiation.
2. **Guided-mode data and logs live in the browser** (IndexedDB) and the user can edit them. Local receipts are labeled "browser only".
3. **Server logs are application records**, not tamper-proof evidence. The evidence content hash is an integrity aid, not an attestation.
4. **Prompt-injection resistance is structural, not linguistic.** The model may still *say* misleading things. Financial numbers shown in cards come from tools, but free-text replies are model output and are labeled as such.
5. **Quota counters use fixed windows.** Bursts at window edges can reach up to twice the limit briefly. Global caps bound the total.
6. **Free hosting.** Render cold starts (~50 s) and provider free-tier limits can cause timeouts. Availability is not guaranteed.
7. **In-memory fallback.** If `DATABASE_URL` is missing the API runs non-durably. Health reports `durableStorage:false`; production must set Postgres.
8. **No automated dependency vulnerability gate in CI** beyond `npm audit` run manually (0 vulnerabilities at the time of writing). Dependabot or CodeQL are recommended next steps.
9. **Not tested:** live Gemini behavior in CI (by design, to avoid paid calls), load testing, and penetration testing by a third party.

4. Data handling

- No passwords, identity documents, payment credentials or bank details are collected. The UI asks users not to enter them.
- Sessions expire after 120 minutes by default. `DELETE /v1/session` deletes all of a session's rows (cascade). Expired sessions are purged opportunistically. Provider-side logs and database backups

follow the providers' own retention policies, and deleting a session does not guarantee immediate removal from them.
- There are no analytics, trackers or cookies.

See TEST_RESULTS.md for the exact test runs.